Finding All Tandem Arrays in DNA Sequences

Jian-Guo Chen and Chia-Tung Lee
Computer Science and Information Engineering Department
National Chi-Nan University
rctlee@ncnu.edu.tw

Abstract

A tandem array is a substring of the form x^k, where x is any unspecific substring and k is at least two (when k is 2, x^2 is also called a tandem repeat or square). A non-extendable tandem array occurring in string S is a tandem array x^k which are not followed or preceded by another occurrence of x in S. The problem of this thesis is defined as follows: Given a string S of length n, find all non-extendable tandem arrays of S. In the thesis, we present an O(nlogn) algorithm for the problem. The algorithm is based on a variation of an O(nlogn) algorithm for finding all squares in S. Instead of directly finding all non-extendable tandem arrays, we first find all squares and then use them to construct all non-extendable p-periodic substrings whose definition follows: A non-extendable p-periodic substring is a substring Y with maximal length m such that the ith character is equal to the $(i+p)$th character of Y for all $i \in \{1,\ldots,m-p\}$ and $\lfloor m/p \rfloor \geq 2$. With all non-extendable p-periodic substrings found, we can easily determine all non-extendable tandem arrays at once.

1 Introduction

In DNA sequences, there exists a category of consecutively repetitive substrings of the form x^k, where *x* is any unspecified substring, and *k* is an integer with at least two [16]. Such a substring x^k is called a tandem array. For example, abcabcabc of the form abc^3 is a tandem array. When *k* is 2, x^k is also called a tandem repeat or square. For example, aa, abab and abbabb are squares. The problem of finding all tandem arrays (or squares) in a string has been found to play an important role in biology [10, 4, 5, 18, 6, 3, 14]. A number of researches concerning the problem have been published [4, 10, 5, 16, 18, 15, 1, 2, 8]. The work by Stoye and Gusfiled [16] has provided an optimal algorithms for detection of all tandem arrays (or squares) in a string of length *n* using a suffix tree in time *O*(*n*log*n*). Now we study an interesting kind of tandem arrays which is composed of non-extendable tandem arrays occurring in a string *S*, that is, those tandem arrays x^k which are not followed or preceded by another occurrence of *x* in *S*. For example, abababababab is a non-extendable tandem arrays in cababababababc while abababab is not. The set of such tandem arrays represents in a compact way all tandem arrays in *S*. It is known that there may be up to *O*(*n*log*n*) tandem arrays, even if only non-extendable tandem arrays are considered [7, 12, 16]. An optimal algorithm using a suffix tree to find such tandem arrays was proposed in [16]. However, the data structure of suffix trees is hard to be implemented. Therefore, it will be very useful to design an optimal algorithm which is capable of finding all non-extendable tandem arrays in a string using easy-to-implement data structures. In this thesis, we present an easy-to-implement and optimal algorithm to find all non-extendable tandem arrays in a string. The algorithm is based on an *O*(*n*log*n*) algorithm for finding all squares in a string with length *n*, which appeared in [15]. By means of the algorithm, a simple approach can be constructed to find all non-extendable tandem arrays in the same time complexity. In Section 2, a simple approach for finding all non-extendable tandem arrays is presented. Section 3 proves its time complexity in order to hold its optimality and Section 4 shows an example for the simple approach. Finally, we shall conclude our approach and give the future works for it in the last section.

Throughout this thesis, the following notations are used. For a string *S*, s_i denotes the *i*th character of *S* while $s_i s_{i+1} \ldots s_j$ denotes a substring of *S* starting at the *i*th character and ending at the *j*th character. The length of *S* is denoted by $|S|$. Prefixes and Suffixes of a string *S* are $s_1 s_2 \ldots s_i$ and $s_i s_{i+1} \ldots s_{|S|}$ where $1 \leq i \leq |S|$, respectively. For two strings *P* and *Q*, *PQ* denotes a string formed by concatenating *P* and *Q*.

2 A simple approach to detecting all non-extendable tandem arrays

In the section, we present an *O*(*n*log*n*) algorithm for detecting all non-extendable tandem arrays in a string of length *n*. The algorithm is

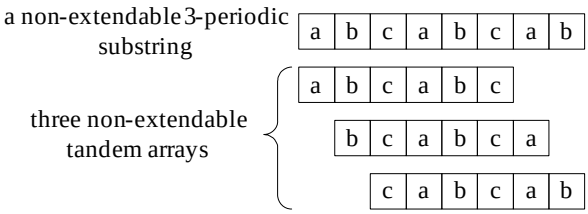

Figure 1 There are three non-extendable tandem arrays abcabc, bcabca and cabcab in a non-extendable 3-periodic substring abcabcab

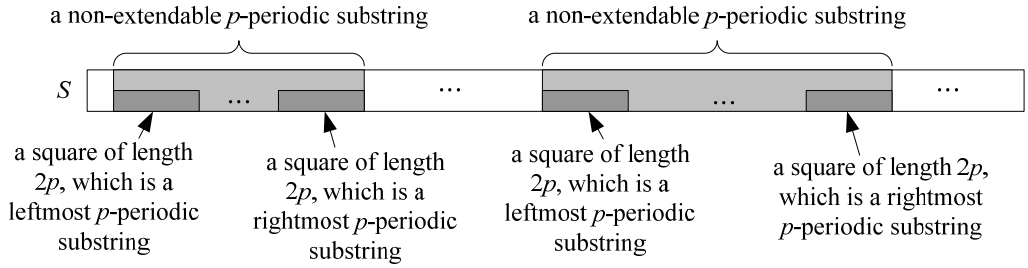

Figure 2 All non-extendable p-periodic substrings of S can be found by extending all squares of length $2p$ from left to right of S

based on a variation of [15]. We don't directly find all non-extendable tandem arrays. Instead, we first find all squares and then use them to construct all non-extendable tandem arrays. The idea is based upon an important observation that squares of length $2p$ with consecutive ending positions in a string S form a p-periodic substring, which is a substring Y with length m such that $y_i = y_{i+p}$ for all $i \in \{1,\ldots,m-p\}$ and $\lfloor m/p \rfloor \geq 2$.

For the example where $S = cababab$, three squares of length 4 ending at 5, 6 and 7 of S, namely $s_2s_3s_4s_5 = abab$, $s_3s_4s_5s_6 = baba$ and $s_4s_5s_6s_7 = abab$, form a 2-periodic substring ababab of S. If the starting position of a p-periodic substring Y is not preceded by that of a square with length $2p$ and the ending position of Y is not followed by that of a square with length $2p$, then Y is a non-extendable p-periodic substring, which is a p-periodic substring Y such that Y becomes not p-periodic as a result of being followed or preceded by a character. For the example where $S = abababdabcabcab$, ababa is a non-extendable 2-periodic substring; abcabcab is a non-extendable 3-periodic substring. With a non-extendable p-periodic substring Y of length m, it can be easily seen that there is a non-extendable tandem array x^k where $|x| = p$ and $k = \lfloor m/p \rfloor$ ending at position i of Y where $p \times \lfloor m/p \rfloor \leq i \leq m$. For the example where a non-extendable 3-periodic substring Y is abcabcab, there are three non-extendable tandem arrays ending at position 6 through 8 of Y: abcabc, bcabca and cabcab, as shown in Figure 1.

The problem is how to find all squares of length $2p$ to construct all non-extendable p-periodic substrings. We now define some terms. A leftmost (rightmost) p-periodic substring is a p-periodic substring Y such that the starting (ending) position of a p-periodic substring Y is not preceded (followed) by that of a square with length $2p$. For the example where a string S is dabcabcabcb, *abcabc* starting at position 2 of S is a leftmost p-periodic substring while bcabca starting at position 3 of S is not; abcabc starting at position 5 of S is a rightmost p-periodic substring while cabcab starting at position 4 of S is not. Suppose that we are given a string $S = s_1s_2\ldots s_n$. By definition, a non-extendable p-periodic substring of S is a leftmost and rightmost substring of S. It is obvious that we can find a non-extendable p-periodic substring of S by extending right a leftmost p-periodic substring, which is also a square of length $2p$, until a rightmost p-periodic substring, which is also a square of length $2p$, is found. This implies that all non-extendable p-periodic substrings of S can be found by extending all squares of length $2p$ from left to right of S, as shown in Figure 2.

All squares of length $2p$ with consecutive ending positions can be found in [15]. However, the order of its recursive calls is a preorder. For the example where a string $S = s_1s_2\ldots s_{16}$, Figure 3 shows the order where a circled number stands for the priority of its recursive calls. Such order causes squares of length $2p$ found recursively not to appear from left to right of S.

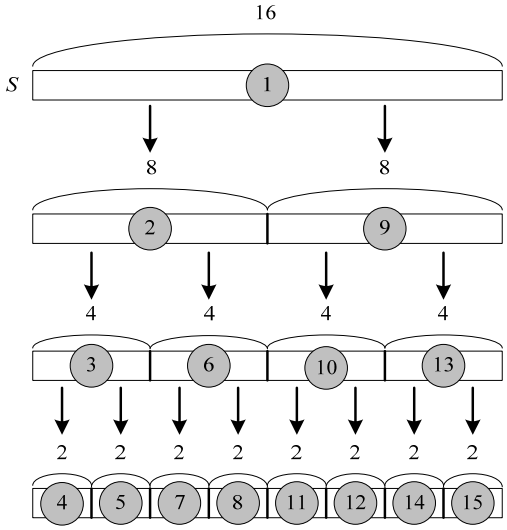

Figure 3 The preorder of the *Findsquares* algorithm

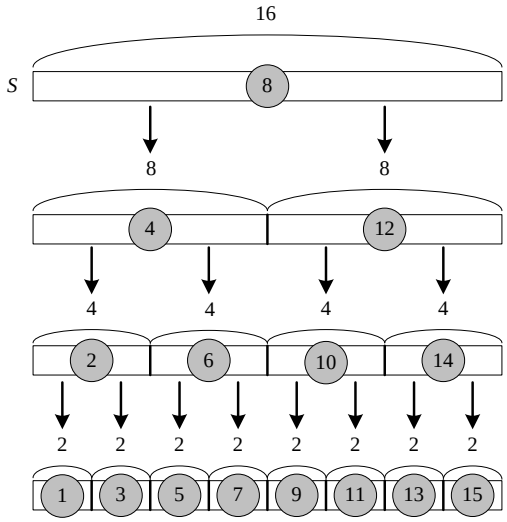

Figure 4 The in-order of the *Findsquares* algorithm

A trick allows us to make the order of recursive calls appear from left to right of *S*. The trick is to change the preorder into an in-order by exchanging *findsquares*($s_1 s_2 \dots s_{\lceil n/2 \rceil}$) and *newsquare*($s_1 s_2 \dots s_{\lceil n/2 \rceil}$, $s_{\lceil n/2 \rceil+1} s_{\lceil n/2 \rceil+1} \dots s_n$) in [15]. For the example of Figure 3, its in-order is shown in Figure 4. With the in-order, we can make the squares of length 2*p* found recursively appear from left to right of *S*. The problem is how to extend from left to right of *S* the squares of length 2*p* to construct non-extendable *p*-periodic substrings. We now introduce two auxiliary tables for constructing non-extendable *p*-periodic substrings. Suppose we are given a string $S = s_1 s_2 \dots s_n$. The first table is called the Start Table of *S* and the second one is called the End Table of *S*. The entries of the Start Table and the End Table of *S* are denoted as *START*[*p*]'s and *END*[*p*]'s, respectively, where $1 \le p \le \lfloor n/2 \rfloor$. For each *p*, we store the starting position and the ending position of a current leftmost *p*-periodic substring of *S* into *START*[*p*] and *END*[*p*], respectively. With these two tables, we can easily determine whether a current leftmost *p*-periodic substring is a rightmost *p*-periodic substring. Suppose that there are squares of length 2*p* with consecutive ending positions appearing after the current leftmost *p*-periodic substring.

In other words, there is a *p*-periodic substring *Y* appearing after the current leftmost *p*-periodic substring. There are two cases:

Case 1: *END*[*p*] is not followed by the ending position of the leftmost square *Q* with length 2*p* in *Y*.

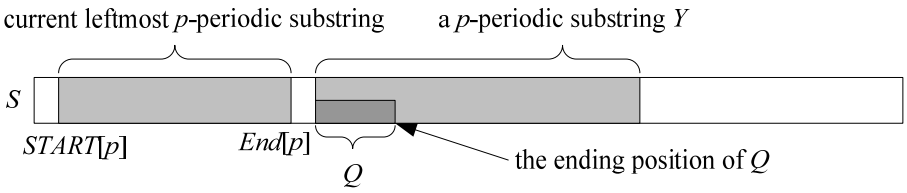

Figure 5 Case 1 where *END*[*p*] is not followed by the ending position of the leftmost square with length 2*p* in *Y*

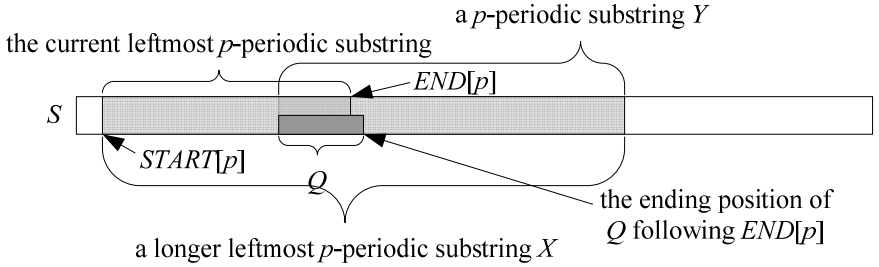

Figure 6 Case 2 where *END*[*p*] is followed by the ending position of the leftmost square *Q* with length 2*p* in *Y*

This case, shown in Figure 5, means that the current leftmost *p*-periodic substring is a rightmost *p*-periodic substring. In other words, it is a non-extendable *p*-periodic substring. At this time, we first report that there is a non-extendable *p*-periodic substring. Then, *START*[*p*] and *END*[*p*] are updated with the starting position and the ending position of *Y*, respectively.

Case 2: *END*[*p*] is followed by the ending position of the leftmost square *Q* with length 2*p* in *Y*

This case is shown in Figure 6. It can easily be seen that the current leftmost *p*-periodic substring can be extended a longer leftmost *p*-periodic substring, that is, a leftmost *p*-periodic substring *X*, which starts at *START*[*p*] and ends at the ending position of *Y*. Now *X* becomes the current leftmost *p*-periodic substring. Thus, we update *END*[*p*] with the ending position of *Y*.

If there doesn't exist a *p*-periodic substring *Y*. appearing after the current leftmost *p*-periodic substring, then we directly report a non-extendable *p*-periodic substring which starts at *START*[*p*] and ends *END*[*p*]. The following shows the algorithm for constructing all non-extendable *p*-periodic substrings in a string $S = s_1 s_2 \ldots s_n$, denoted by Algorithm *Findperiodics*. In the wake of the *Findperiodics* algorithm, we list the *newperiodic*(*P*, *Q*, *d*) algorithm where *P* and *Q* are substrings of *S*; *d* is the ending position of *P* with respect to *S* . The *newperiodic*(*P*, *Q*, *d*) algorithm extends the (*P*, *Q*)-squares of length 2*p* with consecutive ending positions of *S* to construct non-extendable *p*-periodic substrings.

Algorithm *Findperiodics*

Input: a string $S = s_1 s_2 \ldots s_n$.
Output: all non-extendable *p*-periodic substrings in *S*.

START[*p*] = 0 and *END*[*p*] = 0 for all *p* where $1 \le p \le \lfloor n/2 \rfloor$
function *findperiodics*(*S*){
  if $(|S| > 1)$ {
    findperiodics($s_1 s_2 \ldots s_{\lceil n/2 \rceil}$);
    newperiodic($s_1 s_2 \ldots s_{\lceil n/2 \rceil}$, $s_{\lceil n/2 \rceil+1} s_{\lceil n/2 \rceil+1} \ldots s_n$, $\lceil n/2 \rceil$);
    findperiodics($s_{\lceil n/2 \rceil+1} s_{\lceil n/2 \rceil+1} \ldots s_n$);
  }
}
for $(p = 1; p \le \lfloor n/2 \rfloor; p++)$ {
  if(*START*[*p*] != 0){
    There is a non-extendable *p*-periodic substring starting at *START*[*p*] and ending at *END*[*p*].
  }
}

Algorithm *newperiodic*(*P, Q, d*)

Let the Prefix Table of *Q*, the Suffix Table for *P* and *Q*, the Prefix Table of P^r (the reverse of *P*) and the Suffix Table for Q^r (the reverse of *Q*) and P^r be $PREF_Q[i]$'s, $SUFF_P[i]$'s, $PREF_{P^r}[i]$'s and $SUFF_{Q^r}[i]$, respectively.

Input: a string *P* of length *k*, a string *Q* of length *m* and an integer *d*.
Output: partial non-extendable *i*-periodic substrings for all *i* where $1 \le i \le \min(k, m)$.

```
function newperiodic(P, Q, d){ // start of function
    Construct PREF_Q[i]'s, SUFF_P[i]'s, PREF_{P^r}[i]'s and SUFF_{Q^r}[i].
    PREF_Q[m + 1] = 0;
    PREF_{P^r}[k + 1] = 0;
    for (i = 1; i ≤ min(k, m); i++){ //start of for loop
        rfirst = d + 2i − SUFF_P[i];
        rlast = d + min(2i − 1, i + PREF_Q[i + 1]);
        lfirst = d − min(2i − 1, i + PREF_{P^r}[i + 1]) + 2i;
        llast = d − (2i − SUFF_{Q^r}[i]) + 2i;
        if( lfirst ≤ llast = rfirst ≤ rlast ){
            if(START[i] = 0 and END[i] = 0){START[i] = lfirst; END[i] = rlast;}
            else{
                if(lfirst = END[i] + 1){END[i] = rlast;}
                else{
                    There is a non-extendable i-periodic substring starting at START[i] and ending at END[i].
                    START[i] = lfirst; END[i] = rlast;
                }
            }
        }else{ // start of else
            if (lfirst ≤ llast){
                if(START[i] = 0 and END[i] = 0){START[i] = lfirst; END[i] = llast;}
                else{
                    if(lfirst = END[i] + 1){
                        There is a non-extendable i-periodic substring starting at START[i] and ending at llast.
                    }else{
                        There is a non-extendable i-periodic substring starting at START[i] and ending at
                        END[i].
                        There is a non-extendable i-periodic substring starting at lfirst and ending at llast.
                    }
                    START[i] = 0; END[i] = 0;
                }
            }
            if (rfirst ≤ rlast) {
                if(START[i] = 0 and END[i] = 0){START[i] = rfirst; END[i] = rlast;}
                else{
                    There is a non-extendable i-periodic substring starting at START[i] and ending at END[i].
                    START[i] = rfirst; END[i] = rlast;
                }
            }
        } // end of else
    } // end of for loop
} // end of function
```

The above algorithm is easily modified to report non-extendable tandem arrays in non-extendable *p*-periodic substrings. The whole algorithm for finding all non-extendable tandem arrays, denoted by Algorithm *Findtarrays*, is shown in the following.

Algorithm *Findtarrays*

Input: a string $S = s_1 s_2 \dots s_n$.
Output: all non-extendable tandem arrays in *S*.

START[*p*] = 0 and *END*[*p*] = 0 for all *p* where $1 \le p \le \lfloor n/2 \rfloor$
function *findtarrays*(*S*){
  if $(|S| > 1)$ {
    findtarrays($s_1 s_2 \dots s_{\lceil n/2 \rceil}$);
    newtarrays($s_1 s_2 \dots s_{\lceil n/2 \rceil}$, $s_{\lceil n/2 \rceil+1} s_{\lceil n/2 \rceil+1} \dots s_n$, $\lceil n/2 \rceil$);
    findtarrays($s_{\lceil n/2 \rceil+1} s_{\lceil n/2 \rceil+1} \dots s_n$);
  }
}
for $(p = 1; p \le \lfloor n/2 \rfloor; p++)$ {
  if(*START*[*p*] != 0){
    There are non-extendable tandem arrays of length $p \times \lfloor (END[p] - START[p] + 1)/p \rfloor$ ending at $START[p] + p \times \lfloor (END[p] - START[p] + 1)/p \rfloor$ through *END*[*p*].
  }
}

It is apparent that the time complexity of the *newtarrays*(*P*, *Q*, *d*) algorithm is $O(|PQ|)$ because only one **for** loop with some **if** conditions inside.

3 The analysis of time complexity

Suppose that we are given an input $S = s_1 s_2 \dots s_n$. The *Findtarrays* algorithm is combined with three parts: initializing the Start and the End Table, one **for** loop for reporting non-extendable tandem arrays and the *findtarrays*(*S*) function. Obviously, the time complexity of initializing two tables and the **for** loop is $O(n)$. The time complexity of the *findtarrays*(*S*) function is proved in the following.

Let $T(n)$ be the time that the *findtarrays*(*S*) function of the *Findtarrays* algorithm takes. $T(n)$ results in the following recurrence:

$$T(1) = b$$
$$T(n) = T(n/2) + cn + T(n/2)$$

for some constants *b* and *c*. The *cn* is from *newtarrays*($s_1 s_2 \dots s_{\lceil n/2 \rceil}$, $s_{\lceil n/2 \rceil+1} s_{\lceil n/2 \rceil+1} \dots s_n$, $\lceil n/2 \rceil$) while $2T(n/2)$ results from the two recursive calls, namely *findtarrays*($s_1 s_2 \dots s_{\lceil n/2 \rceil}$) and *findtarrays*($s_{\lceil n/2 \rceil+1} s_{\lceil n/2 \rceil+1} \dots s_n$). By reasoning $T(n)$, we obtain

$$\begin{aligned}
T(n) &= T(n/2) + cn + T(n/2) = cn + 2T(n/2) \\
&= cn + 2(cn/2) + 4T(n/4) \\
&= cn + 2(cn/2) + 4(cn/4) + 8T(n/8) \\
&= 2^0(cn/2^0) + 2^1(cn/2^1) + 2^2(cn/2^2) \\
&+ 2^3(cn/2^3) + \dots + 2^{\log_n - 1}(cn/2^{\log_n - 1}) \\
&+ 2^{\log_n} T(n/2^{\log_n}) \\
&= 2^0(cn/2^0) + 2^1(cn/2^1) + 2^2(cn/2^2) \\
&+ 2^3(cn/2^3) + \dots + 2^{\log_n - 1}(cn/2^{\log_n - 1}) + 2^{\log_n} T(1) \\
&= 2^0(cn/2^0) + 2^1(cn/2^1) + 2^2(cn/2^2) \\
&+ 2^3(cn/2^3) + \dots + 2^{\log_n - 1}(cn/2^{\log_n - 1}) + nd \\
&= cn \log n + nd \\
&= O(n \log n)
\end{aligned}$$

Therefore, the time complexity of Algorithm *Findtarrays* is $O(n \log n)$.

4 An example for the simple approach

Suppose that a string *S* is cbcabcabcababcab. At first, we set *START*[*i*] and *END*[*i*] to be zero for all *i* where $1 \le i \le \lfloor 16/2 \rfloor$, as shown in Table 1 and 2.

Table 1 The Start Table of *S* = cbcabcabcababcab

i	1	2	3	4	5	6	7	8
START[*i*]	0	0	0	0	0	0	0	0

Table 2 The End Table of *S* = cbcabcabcababcab

i	1	2	3	4	5	6	7	8
END[*i*]	0	0	0	0	0	0	0	0

Next, we start to find (*P*, *Q*)-squares in $PQ = cb$

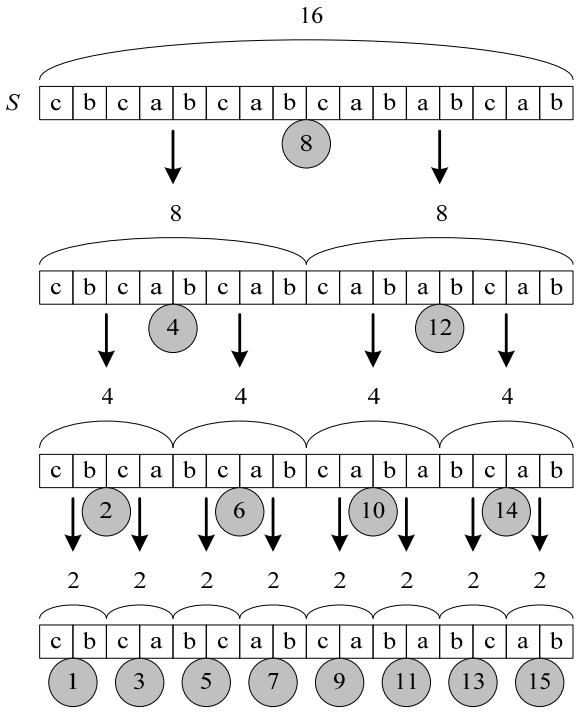

Figure 7 The inorder of the *Findsquares* algorithm for *S* = cbcabcabcababcab

where $\boldsymbol{P = s_1}$ and $\boldsymbol{Q = s_2}$. In this case, no (*P*, *Q*)-square is found in $\boldsymbol{s_1 s_2 = cb}$. The reason that $\boldsymbol{s_1 s_2 = cb}$ is the first substring to be processed is from the in-order of the approach shown in Figure 7.

As a result, the second substring to be processed is $\boldsymbol{s_1 s_2 s_3 s_4 = cbca}$. This substring shows no (*P*, *Q*)-square where $\boldsymbol{P = s_1 s_2}$ and $\boldsymbol{Q = s_3 s_4}$. We keep finding (*P*, *Q*)-squares in $\boldsymbol{PQ = ca}$ where $\boldsymbol{P = s_3}$ and $\boldsymbol{Q = s_4}$. There exits no (*P*, *Q*)-square. When the substring $\boldsymbol{s_1 s_2 s_3 s_4 s_5 s_6 s_7 s_8 = cbcabcab}$ is processed, two squares of length 6 ending at position 7 through 8 of *S*, namely $\boldsymbol{s_2 s_3 s_4 s_5 s_6 s_7}$ and $\boldsymbol{s_3 s_4 s_5 s_6 s_7 s_8}$, form a current leftmost 3-periodic substring, which starts at position 2 and ends at position 8 of *S*. Hence, Table 1 and 2 are updated with *START*[3] = 2 and *END*[3] = 8, as shown below:

The Start Table of *S* = cbcabcabcababcab

i	1	2	3	4	5	6	7	8
START[*i*]	0	0	2	0	0	0	0	0

The End Table of *S* = cbcabcabcababcab

i	1	2	3	4	5	6	7	8
END[*i*]	0	0	8	0	0	0	0	0

Conforming to the in-order in Figure 7, we can see that no (*P*, *Q*)-square is found in $\boldsymbol{s_5 s_6 = bc}$, $\boldsymbol{s_5 s_6 s_7 s_8 = bcab}$ and $\boldsymbol{s_7 s_8 = ab}$. However, when $\boldsymbol{s_1 s_2 s_3 \cdots s_{16} = cbcabcabcababcab}$ is met, we find three squares of length 6 ending at position 9 through 11 of *S*. At the time, *END*[3] is examined in order to extend the current 3-periodic substring into a longer one. In this case, we find that *END*[*i*] + 1 is equal to 9. This means that the current 3-periodic substring starting at *START*[3] and ending at *END*[3] can be extended a longer one starting at *START*[3] and ending at 11, namely $\boldsymbol{s_2 s_3 s_4 s_5 s_6 s_7 s_8 s_9 s_{10} s_{11} = bcabcabcab}$. At the same time, *END*[3] is updated with 11. Now Table 3-2 becomes:

The End Table of *S* = cbcabcabcababcab

i	1	2	3	4	5	6	7	8
END[*i*]	0	0	11	0	0	0	0	0

Likewise, a leftmost 2-periodic substring starting at position 10 and ending at position 13, namely $\boldsymbol{s_{10} s_{11} s_{12} s_{13} = abab}$ is found in $\boldsymbol{s_9 s_{10} s_{11} s_{12} s_{13} s_{14} s_{15} s_{16} = cbabcab}$. Plus, the Start and the End Table are updated with *START*[2] = 10 and *END*[2] = 13, as shown in the following.

The Start Table of *S* = cbcabcabcababcab

i	1	2	3	4	5	6	7	8
START[*i*]	0	10	2	0	0	0	0	0

The End Table of *S* = cbcabcabcababcab

i	1	2	3	4	5	6	7	8
END[*i*]	0	13	11	0	0	0	0	0

Finally, it is reported that there are two tandem arrays of length 9 ending at position 10 through 11 of *S* and one tandem array of length 4 ending at position 13, as follows:

1. $s_2 s_3 s_4 s_5 s_6 s_7 s_8 s_9 s_{10} = bcabcabca$
2. $s_3 s_4 s_5 s_6 s_7 s_8 s_9 s_{10} s_{11} = cabcabcab$
3. $s_{10} s_{11} s_{12} s_{13} = abab$.

5 Conclusions and future works

Given a string *S* of length *n*, we present a simple approach without suffix trees to find all non-extendable tandem arrays in the same time complexity $O(n \log n)$ as an optimal algorithm, in which suffix trees are used. Our approach is based on a variation of an $O(n \log n)$ algorithm without suffix trees to find all squares in *S*. We first find all squares and then use them to construct all non-extendable tandem arrays. Instead of using suffix trees, we only take advantage of simple data structure, making our approach not only simple but also practical.

In DNA sequences, there exist many kinds of repeats, such as, primitive tandem arrays [12], palindromes and approximate repeats. By constructing different auxiliary tables, we hope to modify with a little effort our approach to solve most problems closely concerned with repeating groups.

References

[1] Frequency-Domain Analysis of Biomolecular Sequences, Anastassiou, D., Bioinformatics, Vol. 16, 2000, pp. 1073-1081.

[2] Genomic Signal Processing, Anastassiou, D., IEEE Signal Processing Magazine, Vol. 18, 2001, pp. 8-20.

[3] Hypermutable Minisatellites, a Human Affair?, Bois, P. R. J., Genomics, Vol. 81, 2003, pp. 349-355.

[4] Tandem Repeats Finder: a Program to Analyze DNA Sequences, Benson, G., Oxford University Press, Vol. 27, 1999, pp. 573-580.

[5] Detection and Visualization of Tandem Repeats in DNA Sequences, Buchner, M. and Janjarasjitt, S., IEEE Transactions on Signal Processing, Vol. 51, 2003, pp. 2280-2287.

[6] Comparison of Minisatellites, Berard, S. and Rivals, E., Journal of Computational Biology, Vol. 10, 2003, pp. 357-372.

[7] An Optimal Algorithm for Computing the Repetitions in a Word, Crochemore, M., Information Processing Letters, Vol. 12, 1981, pp. 244-250.

[8] Jewels of Stringology, Crochemore, M. and Rytter, W., World Scientific Publishing Company, 2003.

[9] How Many Squares Can a String Contain?, Fraenkel, A. S. and Simpson, J., Journal of Combinatorial Theory, Vol. 82, 1998, pp. 112-120.

[10] Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology, Gusfield, D., Cambridge University Press, 1997.

[11] Linear Time Algorithms for Finding and Representing All the Tandem Repeats in a String, Gusfield, D. and Stoye, J., Journal of Computer and System Sciences, Vol. 69, 2004, pp. 525-546.

[12] Finding Maximal Repetitions in a Word in Linear Time, Kolpakov, R. and Kucherov, G., IEEE 40th Annual Symposium on Foundations of Computer Science, 1999, pp. 596-604.

[13] Fast Pattern Matching in Strings, Knuth, D. E., Morris, J. H. and Pratt, V. R., SIAM Journal on Computing, Vol. 6, 1977, pp. 323-350.

[14] Understanding the Human Genome, Moore, S. K., IEEE Spectrum, Vol. 37, 2000, pp. 33-35.

[15] An $O(n\log n)$ Algorithm for Finding All Repetitions in a String, Main, M. and Lorentz, R., Journal of Algorithms, Vol. 5, 1984, pp. 422-432.

[16] Simple and Flexible Detection of Contiguous Repeats Using a Suffix Tree, Stoye, J. and Gusfield, D., Theoretical Computer Science, Vol. 270, 2002, pp. 843-856.

[17] Periodicity Transforms, Sethares, W. A. and Staley, T. W., IEEE Transactions on Signal Processing, Vol. 47, 1999, pp. 2953-2964.

[18] Finding Approximate Tandem Repeats in Genomic Sequences, Wexler, Y., Yakhini, Z., Kashi, Y. and Geiger, D., Proceedings of the eighth annual international conference on Computational molecular biology, 2004, pp. 223-232.